Introduction to Back-End Development

Web Design – Lecture 7

(C1WD1A/NWD011//VT2026)

HÖGSKOLAN
I BORÅS

Recap

- Last time:
  - JavaScript adds behaviour to web pages
  - The DOM: browser's representation of HTML
  - Events and event listeners
  - Progressive enhancement
- Today: What happens on the server

HÖGSKOLAN
I BORÅS

Agenda

1. How the web works
2. Building & consuming APIs
3. Live Coding

HÖGSKOLAN
I BORÅS

Part 1

How the web works

HÖGSKOLAN
I BORÅS

Client-server architecture

- Client (Browser):
  - Sends requests
  - Displays content
  - Runs front-end code

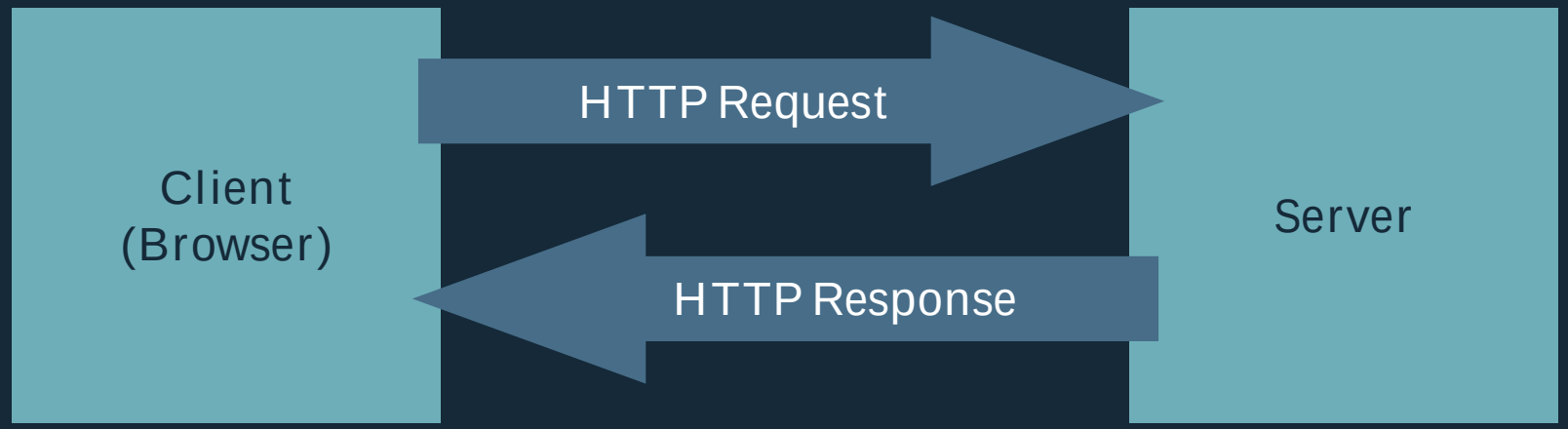
- Server:
  - Receives requests
  - Processes data
  - Sends responses
  - Runs back-end code

HÖGSKOLAN
I BORÅS

HTTP

The protocol

- HyperText Transfer Protocol
- How clients and servers communicate.



HÖGSKOLAN
I BORÅS

HTTP Request

```
GET /about.html HTTP/1.1
Host: www.example.com
```

- Method: What you want to do
  - (GET, POST, PUT, DELETE)
- Path: What resource you want
- Headers: Extra information

HÖGSKOLAN
I BORÅS

HTTP Response

```
HTTP/1.1 200 OK
Content-Type: text/html

<!DOCTYPE html><html>…</html>
```

- Status code: Did it work?
  - (200 OK, 404 Not Found, 500 Error, etc.)
- Headers: Metadata
- Body: The actual content (can be empty)

HÖGSKOLAN
I BORÅS

Common HTTP methods

Method	Purpose	Example
`GET`	Retrieve data	Load a page
`POST`	Send data	Submit a form
`PUT`	Update data	Edit a profile
`DELETE`	Remove data	Delete an account

HÖGSKOLAN
I BORÅS

Static vs dynamic

Static website

- Server sends files as-is
- Same content for everyone
- Just HTML, CSS, JS files

Dynamic website

- Server generates content on request
- Different content for different users
- Requires server-side code

Why backend?

- Things only a server can do:
  - Store and retrieve data (databases)
  - User authentication (login, accounts)
  - Process payments
  - Send emails
  - Keep secrets (API keys, passwords)
  - Business logic (calculations, rules)

HÖGSKOLAN
I BORÅS

Back-end technologies

- Many languages can run on servers:
  - PHP — WordPress, Facebook (originally)
  - Python — Django, Flask
  - JavaScript — Node.js
  - Ruby — Rails
  - Java — Spring
  - C# — ASP.NET Core ← We'll use this

HÖGSKOLAN
I BORÅS

What is .NET?

- A platform from Microsoft:
  - Runtime (executes code)
  - Libraries (pre-built functionality)
  - Tools (for building applications)

- Open source, cross-platform (Windows, Mac, Linux)

ASP.NET Core

- A framework for building web applications with .NET
  - Fast and modern
  - Used by major companies
  - Great tooling (Visual Studio, VS Code)
  - Strong in enterprise environments

- Today we'll use Minimal APIs
  - The simplest way to build an API.

HÖGSKOLAN
I BORÅS

C# in 60 seconds

```
// Variables (note: types are required)
string name = "Anna";
int age = 25;
bool isActive = true;

// Arrays
var items = new[] { "Coffee", "Tea", "Juice" };

// Anonymous objects
var product = new { Id = 1, Name = "Coffee", Price = 35 };
```

HÖGSKOLAN
I BORÅS

Part 2

Building & consuming APIs

HÖGSKOLAN
I BORÅS

What is an API?

- *A*pplication *P*rogramming *I*nterface
  - A way for programs to talk to each other.

- Web API: An API accessed over HTTP.

HÖGSKOLAN
I BORÅS

Why APIs?

- Separate front-end from back-end
- Multiple clients (web, mobile, desktop) share one back-end
- Third-party integrations
- Modern architecture

REST

- REpresentational State Transfer

- Standard HTTP methods (GET, POST, DELETE, etc)
- URLs (or ”endpoints”) represent resources

HÖGSKOLAN
I BORÅS

REST

Example endpoints

Method	URL	Action
GET	/api/products	List all products
GET	/api/products/5	Get product #5
POST	/api/products	Create a product
PUT	/api/products/5	Update product #5
DELETE	/api/products/5	Delete product #5

HÖGSKOLAN
I BORÅS

JSON

- JavaScript Object Notation
- How APIs usually send data:

```
{
  "id": 1,
  "name": "Coffee",
  "price": 35,
  "inStock": true
}
```

HÖGSKOLAN
I BORÅS

JSON arrays

```
[
  { "id": 1, "name": "Coffee", "price": 35 },
  { "id": 2, "name": "Tea", "price": 30 },
  { "id": 3, "name": "Juice", "price": 40 }
]
```

HÖGSKOLAN
I BORÅS

Minimal APIs in ASP.NET Core

- The simplest way to create an API:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/api/hello", () => "Hello, World!");

app.Run();
```

HÖGSKOLAN
I BORÅS

Returning JSON data

```
app.MapGet("/api/products", () => new[] {
    new { Id = 1, Name = "Coffee", Price = 35 },
    new { Id = 2, Name = "Tea", Price = 30 },
    new { Id = 3, Name = "Juice", Price = 40 }
});
```

HÖGSKOLAN
I BORÅS

Route parameters

```
// /api/products/5 => id == 5
app.MapGet("/api/products/{id}", (int id) => {
    return new { Id = id, Name = "Coffee", Price = 35 };
});
```

HÖGSKOLAN
I BORÅS

Handling post requests

```
app.MapPost("/api/products", (Product newProduct) => {
    // Save the product...
    return Results.Created(
        $"/api/products/{newProduct.Id}",
        newProduct
    );
});
```

HÖGSKOLAN
I BORÅS

Returning status codes

```
app.MapGet("/api/products/{id}", (int id) => {
    var product = FindProduct(id);

    if (product is null) {
        return Results.NotFound();
    }

    return Results.Ok(product);
});
```

HÖGSKOLAN
I BORÅS

Connecting front-end to back-end

The Fetch API

- JavaScripts way to make API requests:

```
fetch('/api/products')
  .then(response => response.json())
  .then(data => {
    console.log(data);
  });
```

HÖGSKOLAN
I BORÅS

Fetch with async/await

- Cleaner syntax:

```
async function getProducts() {
  const response = await fetch('/api/products');
  const products = await response.json();
  console.log(products);
}

getProducts();
```

HÖGSKOLAN
I BORÅS

Displaying API data

```
async function loadProducts() {
  const response = await fetch('/api/products');
  const products = await response.json();

  const list = document.querySelector('#product-list');

  products.forEach(product => {
    const li = document.createElement('li');
    li.textContent = `${product.name}: ${product.price} kr`;
    list.appendChild(li);
  });
}
```

HÖGSKOLAN
I BORÅS

Sending data with POST

```
async function addProduct(name, price) {
  const response = await fetch('/api/products', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({ name, price })
  });

  const newProduct = await response.json();
  console.log('Created:', newProduct);
}
```

HÖGSKOLAN
I BORÅS

Error-handling

```
async function getProducts() {
  try {
    const response = await fetch('/api/products');
    if (!response.ok)
      throw new Error('Failed to fetch products');

    const products = await response.json();
    displayProducts(products);
  } catch (error) {
    console.error('Error:', error);
  }
}
```

HÖGSKOLAN
I BORÅS

Part 3

Live coding

HÖGSKOLAN
I BORÅS

Live Coding

Let's create and consume an API

- We'll create a minimal API for a café website:
  - A minimal API with one endpoint that returns a list of menu items
  - A front-end to fetch and display the menu items

HÖGSKOLAN
I BORÅS

The API

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddCors(); // Enable CORS
var app = builder.Build();
// Allow requests from our front-end
app.UseCors(policy => policy.AllowAnyOrigin().AllowAnyHeader());

// Our menu data
var menuItems = new[] {
    new { Id = 1, Name = "Espresso", Price = 30 },
    new { Id = 2, Name = "Cappuccino", Price = 45 },
};
// GET endpoint - returns all items as JSON
app.MapGet("/api/menu", () => menuItems);
app.Run();
```

HÖGSKOLAN
I BORÅS

The HTML

index.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Menu</title>
</head>
<body>
    <h1>Our Menu</h1>
    <ul id="menu-list">
        <li>Loading...</li>
    </ul>
    <script src="script.js"></script>
</body>
</html>
```

HÖGSKOLAN
I BORÅS

The JavaScript

script.js

```
async function loadMenu() {
    const response = await fetch('http://localhost:5000/api/menu');
    const items = await response.json();
    const list = document.querySelector('#menu-list');
    list.innerHTML = '';

    items.forEach(item => {
        const li = document.createElement('li');
        li.textContent = `${item.name} - ${item.price} kr`;
        list.appendChild(li);
    });
}

loadMenu();
```

HÖGSKOLAN
I BORÅS

Part 4

Conclusion

HÖGSKOLAN
I BORÅS

Key takeaways

- Client-server: browser requests, server responds
- HTTP: the protocol for web communication
- APIs: how programs communicate over the web
- JSON: the common data format
- Minimal APIs: the simplest way to build an API in .NET
- Fetch: JavaScript's way to call APIs

HÖGSKOLAN
I BORÅS

Looking ahead

- Next lecture: Course Summary & Usability Testing
  - Review of everything we've covered
  - Practical usability testing techniques
  - Preparing for the exam and project

HÖGSKOLAN
I BORÅS

Questions?

HÖGSKOLAN
I BORÅS